PT

PT's Tactical Renovations
Your Home, Our Mission.

Tactical Foreman — Build Handoff

Prepared by Claude (Opus 5) for the next agent picking this up · 11 August 2026
Repository `plthompsonjr-IAM/animated-enigma` · branch `claude/tactical-foreman-build-m7i3ng` · PR #6

Read this first

This document is written to be accurate rather than flattering. Where something is built but **unverified**, it says so. Where a number is quoted, it was measured, not estimated. Please keep that standard — the entire value of this system to Patrick is that its numbers can be trusted, and a confident claim that turns out to be wrong costs more than an admitted gap.

1. Where the build stands

29 / 50	584	160	47
TASKS COMPLETE	UNIT TESTS	RLS ASSERTIONS	TABLES, ALL RLS-FORCED

Tasks 1–29 are complete, tested, committed, pushed, and applied to the live Supabase project (`zhlkfuvscnblkkyfticz`). Typecheck, lint, and the production build are clean. Migrations through `0043` are applied and verified on the live database.

AREA	STATE	NOTES
Identity, tenancy, RBAC	DONE	9 roles, 40+ permissions, multi-tenant RLS enforced at the database
Leads → clients → projects	DONE	Full CRM pipeline with conversion
Scopes, estimates, proposals	DONE	Immutable version snapshots; client never sees cost or margin
E-signature	DONE	Append-only at the DB level, including against the app's own role
Contracts, change orders	DONE	Revised value derived, never mutated; freeze triggers on signed records
Invoices, payments	DONE	Allocations, aging, project budget rollup
Scheduling	DONE	Calendar-day work items, crew assignment, cross-job conflict detection
Field tasks, punch lists	DONE	Dependencies with blocking derived at read time
Daily logs	DONE	Controlled edit window + append-only revision history

AREA	STATE	NOTES
Photos & documents	UNVERIFIED	Built; private bucket created; **no upload has ever been performed**
Dashboard, financials	DONE	Triage-first dashboard; A/R aging; real margin
Job costing	DONE	Time + expenses; margin from finished jobs only
AI Foreman	NOT STARTED	The only remaining placeholder screen

2. What is NOT verified — do not assume otherwise

File upload has never been exercised end to end

The `project-files` bucket exists and is private. The upload code, validation, signed-URL minting, and org-scoped path constraint are all written and unit-tested. **But no actual file has been pushed through the path.** Treat this as unproven until someone uploads a photo from a real project page and sees it render. This is the single most likely place a latent bug is sitting.

No end-to-end browser testing exists anywhere

Every test is a unit test or a database assertion. No flow has been driven through a browser: not signup, not the client proposal link, not e-signature, not clock-in. The unit tests are thorough and the database rules are proven, but the wiring between them is only proven by the production build compiling. See the Playwright recommendation in §6.

Email delivery does not exist

Every client-facing link — proposal, change-order approval, invoice — is generated correctly and then must be **copy-pasted by hand**. Nothing is sent. This is the largest functional gap in the product and it is blocked on an API key.

3. Blocked on Patrick — nothing moves without these

ITEM	BLOCKS	WHY IT MATTERS
Hourly cost rates per person Settings → Team	All margin reporting	Burdened cost to the business, not wages. Until set, labour is not costed and every margin is understated. The app deliberately shows nothing rather than guessing — a made-up rate would poison every job invisibly.
Email provider API key Resend or Postmark	All client communication	Without it the product cannot send anything to a client.
Attorney review of contract terms	Sending any contract	The starter template carries visible `[BRACKETED]` blanks and the app warns while any remain. The **Ohio-specific right-to-cancel wording is the priority item** — it is state-regulated and the current text is a placeholder.
Payment processing decision	Client self-service payment	Payments are currently recorded by hand. No card processing, no client portal payment.

4. Architecture you must not break

The security model — read this before touching the database

The application connects to Postgres as a role with `BYPASSRLS`. This means `ALTER TABLE … ENABLE ROW LEVEL SECURITY` does *nothing* on its own.

Every tenant table needs **`ENABLE` and `FORCE`**. A table with only `ENABLE` leaks one contractor's jobs into another contractor's account, silently, with no error anywhere. `scripts/test-setup-sql.sh` fails the build if any table is missing either. Never disable that check.

The shape every feature follows

Do not invent a new structure. Every domain is built the same way, in this order:

1. `src/lib/<domain>/<domain>-core.ts` — pure logic, zero I/O. **This is where the real decisions live.** Build it first and test it exhaustively.
2. `<domain>-core.test.ts` — happy path, every rejection, both boundaries, degenerate input.
3. `queries.ts` — reads, always org-scoped.
4. `actions.ts` — `'use server'` writes behind a permission gate.
5. `src/components/<domain>/` — presentation.
6. Migrations + RLS assertions.

Conventions that hold everywhere

- **Money is `numeric(14,2)`, hours `numeric(12,4)`.** Never floats.
- **A calendar day is `date`, not `timestamptz`.** A crew frames Tuesday through Friday; storing that as an instant makes the day shift with whoever reads it. This has already caused one real bug.

- **Say nothing rather than zero.** A job with no contract has `null`, not `0`. Null renders as "—"; zero renders as a lie.
- **Push invariants into the database** when a query would otherwise have to defend against them: derived values by trigger, freeze triggers on issued records, append-only for evidence, exclusion constraints for overlap.
- **Gate at the query, not the render.** Someone without `financials:read` should never have the amounts *fetched*.
- **The nav is fixed at twelve sections.** Do not add to it.

Judgement calls already made — please honour them

- **Cost-to-date on a running job is not margin.** A job 30% built and 50% billed looks wonderful and isn't. Company margin is computed from *finished jobs only*.
- **Contract value is never mutated.** Revised value is derived as original + approved change orders.
- **Nothing is client-visible by default.** A photo of an open wall is internal until somebody decides otherwise.
- **Daily logs lock after a controlled window.** Corrections go in a later log; the record is never rewritten. This is what makes it hold up in a delay claim.

5. What to build next

TASK	WHAT	NOTES
30	AI Foreman	The last placeholder. It now has real material: schedule conflicts, cost overruns, aging receivables, log gaps, blocked tasks. **Every AI action must be a reviewable draft, never an autonomous write.** Cite sources from the job record.
31–33	Email + notifications	Provider integration, templates, approval-gated sending, per-user preferences. Unblocks the biggest functional gap.
34–36	Client & subcontractor portals	Strict data boundaries. The `client_visible` flags and portal RLS policies are already designed in `docs/database-schema.md §8.4`.
37–40	Subcontractors, reports, exports, audit log	CSV/PDF export; audit history.
41–45	Settings, security review, automated test suite	This is where E2E testing belongs.
46	Production deployment	SSL, backups, error monitoring, rollback procedure, launch checklist.
47–50	Voice field ops, photo AI, permits, warranty	Post-MVP. Photo AI needs explicit "this is not an inspection" disclaimers.

6. Recommendations

Three skills to carry forward

These already exist in `.claude/skills/`. They encode the parts of this build that were repetitive, fiddly, or security-critical. Read them before writing code.

SKILL	WHAT IT PREVENTS
`domain-slice`	Reinventing the architecture. Encodes the seven-part shape, the pure-core-first order, and the two bug classes that keep surfacing: timezone assumptions and rounding that must sum to 100.
`db-change`	The tenant leak. Covers the two-file migration pattern, the journal/snapshot chaining, the empty `search_path` the Supabase advisor flags, and verifying `FORCE` on the live database rather than assuming it.
`ship-check`	Claiming green when it is red. Includes the SQL-shape smoke test — unit tests never touch a database and the build never runs a query, so a wrong enum value compiles, passes, ships, and throws on first page load. That check has already caught two real bugs.

Three connectors that would move the product furthest

CONNECTOR	PRIORITY	WHY
Resend or Postmark	CRITICAL	Unblocks every client-facing flow at once — proposals, change-order approvals, invoices, reminders. Nothing else in the backlog delivers as much per hour of work. Postmark has better deliverability for transactional mail; Resend has the simpler API. Either is fine; pick one and wire it behind a small `src/lib/email/` module so the provider stays swappable.
Stripe	HIGH	Contractors get paid faster when the client can pay from the invoice. The invoice ledger, allocations, and payment records are already built and tested — Stripe only needs to feed `payments`. Use Payment Links or Checkout first; a full portal can come later. **Note:** the invoice freeze trigger deliberately permits `amount_paid` updates on an issued invoice, so webhook writes will work without loosening anything.
Google Calendar or Twilio SMS	MEDIUM	Calendar: crew assignments already exist as dated work items with people attached — syncing them to phones is a small step with real field value, and `site_visits` already carries a `google_event_id` slot for exactly this. Twilio is the alternative if the crew reality is "nobody reads email" — SMS reaches a jobsite where email does not. Pick based on how Patrick's crew actually communicates, not on which is easier.

Three plugins / tools to add

TOOL	PRIORITY	WHY
Playwright	CRITICAL	The largest hole in the test strategy. 584 unit tests and 160 database assertions prove the pieces; *nothing* proves they are wired together. Start with the four flows where a break is invisible and expensive: **file upload** (never exercised at all), the public proposal link and e-signature, clock-in/clock-out, and invoice issue → payment. Chromium is already installed in the dev container.
Sentry	HIGH	Before Task 46. A contractor on a roof will not file a bug report — they will stop using the app. Server-action errors are currently logged to stdout and vanish. Sentry's Next.js SDK captures both server and client. Scrub the payloads: this app holds client addresses, contract values, and pay-inferable hours.
Vercel + Supabase branching	HIGH	Next.js 15 deploys there with the least friction, and preview deployments per PR would have caught things this build could only catch by reasoning. Pair with Supabase branching so migrations get tested against a real database copy before touching production. Required groundwork for Task 46 regardless of the host chosen.

7. Guardrails already in place

Four hooks live in `.claude/settings.json`. They enforce rules that were otherwise held only by an agent remembering them. **Please leave them on.**

HOOK	EVENT	DOES
`verify-evidence.sh`	Stop	Runs typecheck, lint, tests (+ RLS suite when SQL changed). **Blocks the turn on failure.** Prints the real counts so reports quote measurements rather than assertions.
`check-schema-completeness.sh`	Stop	Names which of the four schema follow-through steps are still missing. Skipping the RLS file is a tenant leak; skipping the setup mirror means the next fresh deploy is silently missing a table.
`guard-push.sh`	PreToolUse	**Denies** pushes to any branch but the designated one, and denies force-pushes.
`session-state.sh`	SessionStart	Reports branch, working state, migration count, and what is blocked on Patrick.

The one rule that matters most

Never invent completed integrations, credentials, or test results. If something was not run, say it was not run. If a migration is sitting unapplied, say so in the commit message — not just in chat, because chat scrolls away and the commit does not. Patrick is a contractor making real financial decisions from these numbers. An admitted gap costs him ten minutes; a confident wrong answer costs him a job.

PT's Tactical Foreman — build handoff · Tasks 1–29 complete · 584 unit tests, 160 RLS assertions, 47 tables all RLS-forced · Repository `plthompsonjr-IAM/animated-enigma`, branch `claude/tactical-foreman-build-m7i3ng`, PR #6 · Every figure in this document was measured at time of writing, not estimated.